Tool Learning and Function Calling for Large Language Model Agents: A Comprehensive Survey

Abstract

The rapid evolution of large language models (LLMs) has catalyzed a fundamental paradigm shift in artificial intelligence, transitioning from static natural language processing systems to dynamic, autonomous agents capable of interacting with complex environments. Central to this transition is the paradigm of tool learning, operationalized predominantly through function calling. By enabling LLMs to interface with external utilities, application programming interfaces (APIs), computational engines, and structured databases, tool learning transcends the inherent limitations of isolated parametric memory. This augmentation allows models to acquire real-time knowledge, execute rigorous mathematical logic, and perform grounded actions in both digital and physical domains. This comprehensive survey provides a systematic examination of the theoretical foundations, methodological advancements, and empirical evaluations characterizing the current landscape of tool learning in LLM agents.

We construct a formal taxonomy that deconstructs the tool-use workflow into four interdependent stages: task planning and decomposition, tool retrieval and selection, task execution and function calling, and response generation with iterative reflection. Through this structured lens, we critically analyze the dominant methodological paradigms, tracing the evolutionary trajectory from tuning-free prompting architectures to Supervised Tool Learning (SFT), and ultimately into the sophisticated realm of Reward-Driven Tool Policy Learning via Reinforcement Learning (RL) and Direct Preference Optimization (DPO). The survey further explores the paradigm of autonomous tool creation, where models act as algorithmic toolmakers. Correspondingly, we review the evolution of evaluation frameworks, mapping the transition from static syntactic assessments to dynamic, multi-turn, and multi-agent benchmarks modeled as partially observable Markov decision processes. Finally, we dissect critical open challenges and future prospects, placing particular emphasis on the severe security implications of function-calling jailbreaks, the emerging necessity for parameter-level tool unlearning, and the extension of tool use into reasoning-intensive multimodal environments. By synthesizing these diverse research vectors, this manuscript establishes a unified conceptual framework to guide the future development of robust, generalizable, and secure tool-augmented AI systems.

Introduction

Modern large language models have demonstrated unprecedented capabilities in language comprehension, reasoning, and generation, driven largely by exponential scaling laws and self-supervised pre-training on vast textual corpora [1]. Despite these profound successes, models relying exclusively on parametric memory remain constrained by fundamental architectural bottlenecks. First, their internal knowledge is temporally frozen at the time of

pre-training, rendering them inherently incapable of accessing contemporaneous information or updating their world models dynamically [2]. Second, purely parametric models frequently falter on tasks requiring precise arithmetic, complex symbolic logic, and structured data manipulation. When forced to extrapolate beyond their training distribution without external verification, these models often generate highly plausible yet factually incorrect outputs—a phenomenon widely documented as hallucination [1]. Finally, traditional language models are confined to a strictly generative interface; they lack the agency to enact verifiable changes in the external world or to seamlessly interoperate with specialized software ecosystems [3].

To systematically mitigate these constraints, the research community has increasingly coalesced around tool learning, a computational paradigm that augments LLMs with the ability to invoke external tools, APIs, and computational environments [4, 5, 16]. Tool learning effectively bridges the semantic flexibility of neural language modeling with the deterministic precision of symbolic computation. By acting as a central cognitive controller, the LLM can route specific sub-problems to specialized expert modules, such as Python interpreters, SQL databases, search engines, and discrete mathematical calculators [6]. The integration of these external utilities yields multifaceted benefits: it facilitates dynamic knowledge acquisition, enhances domain-specific expertise, automates complex digital workflows, and significantly bolsters the robustness and interpretability of responses by grounding model outputs in verifiable execution traces [1].

The practical operationalization of tool learning relies on a mechanism known as function calling. In this schema, an LLM must accurately parse a user's natural language intent, select the appropriate function from an available repository, synthesize the requisite arguments in a highly structured format (typically JSON), and seamlessly ingest the execution results back into its ongoing reasoning context [3, 7]. As the ecosystem of available tools scales from a handful of carefully curated APIs to vast, open-source community repositories containing tens of thousands of heterogeneous plugins, the complexity of managing this interaction grows exponentially [16]. Consequently, research has rapidly evolved beyond initial zero-shot prompting techniques. Contemporary frameworks increasingly leverage specialized Supervised Fine-Tuning (SFT) over synthesized interaction trajectories, alongside sophisticated alignment techniques such as Reinforcement Learning from Human Feedback (RLHF) and Direct Preference Optimization (DPO), to instill robust tool-use policies [8, 17, 35].

This survey systematically reviews the state-of-the-art in tool learning and function calling for LLM agents. We extend prior literature reviews by dissecting emergent methodological phenomena, including the persistent challenge of structural collapse during agentic reinforcement learning, the unique security vulnerabilities intrinsic to structured API outputs, and the urgent requirement for targeted tool unlearning mechanisms. Through a comprehensive synthesis of contemporary methodologies, evaluation frameworks, and critical challenges, this manuscript aims to chart the trajectory of tool-augmented agents toward more reliable, autonomous, and secure artificial general intelligence.

Formalization and Operational Workflow

The process by which an LLM agent utilizes an external tool is inherently sequential, dynamic, and highly interdependent. Formally, multi-turn tool learning is frequently modeled as a partially

observable Markov decision process (POMDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, \mathcal{O}, \mathcal{T}, \mathcal{R}, \mathcal{U})$, where the agent interacts with an environment defined by the available toolset [14]. Across the literature, this interaction is universally conceptualized through a four-stage operational pipeline: task planning, tool retrieval, task execution, and response generation [1].

Task Planning and Decomposition

Before interacting with external environments, an LLM agent must decipher the user's overarching intent and recursively decompose complex, high-level requests into manageable, executable sub-tasks. Task planning serves as the essential cognitive scaffolding for subsequent tool use [1]. The ReAct (Reasoning and Acting) framework pioneered the interleaving of explicit chain-of-thought reasoning traces with specific actions [5]. By generating verbal reasoning steps prior to action selection, the agent can logically induce, track, and dynamically update its action plans, accommodating exceptions encountered during execution. Similarly, macro-architectures such as HuggingGPT utilize the LLM specifically as an orchestrating "brain" to parse user requests into a topological dependency graph of sub-tasks, determining the optimal execution order before dispatching assignments to specific machine learning models hosted on community platforms [6].

However, linear planning is highly susceptible to error compounding. If an initial reasoning trace misaligns with the agent's historical context or overarching objective, the subsequent cascade of tool invocations will inevitably fail. Advanced planning frameworks attempt to decouple the planning phase from execution entirely, requiring the model to formulate a complete directed acyclic graph of sub-tasks prior to any API interaction [16]. This decoupled strategy minimizes latency and token overhead while isolating planning logic from downstream execution errors.

Tool Retrieval and Selection

As the global repository of available tools scales into the tens of thousands—often exhibiting pervasive functional overlap and synonymous naming conventions—it becomes computationally infeasible, and contextually detrimental, to inject all available tool schemas into the LLM's active prompt. This scalability bottleneck necessitates robust tool retrieval and selection mechanisms [18, 20].

Traditional approaches rely heavily on dense retrieval architectures, treating the user query and tool descriptions as vectors embedded within a shared latent space. However, simple bi-encoder similarity searches frequently fail to capture the complex, multi-step reasoning required for accurate tool matching [29]. To significantly enhance retrieval accuracy, contemporary methods increasingly employ LLM-driven query expansion. In frameworks such as those proposed by Kachuee et al., the LLM is leveraged to generate a highly descriptive pseudo-query or explicit functional requirement based on conversational context prior to executing the dense retrieval step, successfully bridging the semantic gap between vague user intents and rigid tool documentation [18, 29].

Furthermore, researchers have identified a critical phenomenon termed "hidden-body asymmetry" in tool routing. While an LLM agent may only require a tool's name and parameter signature to invoke it, retrieving the correct tool from a massive database often requires parsing the full, verbose documentation body. Frameworks like SkillRouter deploy a two-stage

retrieve-and-rerank pipeline where both the initial encoder and a specialized LLM reranker maintain full access to the comprehensive tool text, dramatically reducing false positive selections while operating efficiently [28].

To optimize the retrieval phase directly within the LLM's intrinsic architecture, alternative methodologies treat tools as discrete virtual tokens within the model's vocabulary. Frameworks such as ToolkenGPT and ToolGen allocate specific, trainable embeddings for individual tools. By expanding the output matrix, the LLM bypasses traditional vector-database retrieval entirely, predicting the "toolken" as a standard next-token generation step before seamlessly transitioning into argument generation mode [8, 20].

Retrieval Mechanism	Operational Logic	Primary Strengths	Limitations
Dense Bi-Encoders	Matches query vectors to tool description vectors.	High speed and scalability.	Fails on reasoning-intensive queries; surface-level matching.
Query Expansion	Uses LLM to rewrite/expand user query before dense search.	Bridges semantic gap between vague queries and tool docs.	Adds inference latency prior to retrieval step.
Retrieve-and-Rerank	Initial dense search followed by LLM-based cross-encoder reranking.	Analyzes full documentation body; reduces false positives.	Computationally intensive for large candidate pools.
Vocabulary Augmentation	Learns distinct embeddings ("toolkens") for specific tools.	Eliminates external retrieval; highly parameter efficient.	Difficult to scale dynamically to millions of tools without retraining embeddings.

Task Execution and Function Calling

Following the selection of an appropriate tool, the LLM must synthesize syntactically precise and semantically accurate arguments to invoke the API, a process formally defined as function calling or slot filling [3, 47]. The model acts as a compiler, translating natural language instructions into a strictly formatted output, typically a JSON object or an Abstract Syntax Tree

(AST) representation, which the external runtime environment can execute [10].
Execution introduces a paradigm of extreme strictness; unlike natural language generation where multiple phrasing variations are semantically equivalent, an API call containing a single hallucinated parameter name, an incorrect data type, or omitting a mandatory field will trigger a fatal runtime exception. Foundational models fine-tuned explicitly on vast datasets of API calls, such as Gorilla and ToolLLM, demonstrate exceptional capability in constructing correct inputs, drastically curtailing argument hallucination compared to generalized foundational models [3, 4].
Additionally, advanced agents must adeptly navigate complex execution topologies, including parallel function calling—where multiple independent tools are executed simultaneously to minimize latency—and nested function calling, where the output of one tool serves as a mandatory argument for a subsequent invocation [13, 27].

Response Generation and Reflection

In the terminal stage of the workflow, the external execution environment returns the tool's output back to the LLM agent. This payload may manifest as a discrete numerical value, a complex structured JSON response, or a system error traceback. The agent must parse this observation, abstract the relevant information, and synthesize it into a coherent natural language response that directly resolves the user's initial query [1].
Crucially, if the tool execution fails, robust agentic systems engage a reflection or self-correction loop. The agent analyzes the returned error message (e.g., an HTTP 400 Bad Request or a Python syntax error), iteratively adjusts the tool arguments, and re-executes the function. This closed-loop feedback mechanism is vital for resilient tool use in production environments, as real-world APIs frequently exhibit undocumented constraints, evolving schemas, or transient failures [25, 34].

Methods: Paradigms of Tool Learning

The development and optimization of tool-augmented LLMs can be categorized into four dominant methodological paradigms. These paradigms represent a distinct evolutionary progression, shifting from a reliance on the massive parametric scale of frozen models toward explicit, data-driven optimization of dynamic agentic policies [16].

Prompting as Plug-and-Play

The initial paradigm leverages frozen, pre-trained LLMs, inducing tool use entirely via sophisticated in-context learning and prompt engineering. This approach frames tool schemas, usage constraints, and few-shot demonstrations entirely within the model's system prompt. It relies heavily on the model's emergent zero-shot reasoning capabilities to generate the correct API syntax on the fly [7, 16].
Prominent frameworks in this category, such as Chameleon, construct plug-and-play compositional reasoning systems without necessitating any weight updates. Chameleon utilizes an LLM-based planner to autonomously assemble sequences of off-the-shelf vision models, web scrapers, and heuristic modules based purely on natural language descriptions, achieving state-of-the-art results on knowledge-intensive multimodal reasoning tasks [7].
While the Plug-and-Play paradigm offers immense flexibility and avoids the prohibitive

computational costs of model retraining, it confronts severe scaling bottlenecks. The mandatory inclusion of extensive tool documentation consumes significant portions of the LLM's finite context window. This constraint results in elevated inference latency, exorbitant token costs, and a heightened susceptibility to distraction and catastrophic forgetting, particularly when managing intricate, multi-step tasks across extended prompt lengths [8, 16].

Supervised Tool Learning (SFT)

To overcome the inherent brittleness and inefficiency of in-context learning, the research landscape rapidly transitioned to Supervised Tool Learning. This paradigm focuses on internalizing tool-use syntax and functional logic directly into the model's parametric weights via Supervised Fine-Tuning (SFT) [16].

The foundational architecture in this space, Toolformer, demonstrated that models could effectively teach themselves to utilize tools. By leveraging their own in-context learning capabilities, these models annotated massive language modeling datasets with potential API calls. By executing these calls and utilizing a self-supervised loss to filter for invocations that objectively improved next-token prediction, Toolformer curated a high-quality SFT dataset. Training on this corpus dramatically improved zero-shot downstream performance without degrading the model's core language modeling proficiency [2].

Scaling this concept to enterprise dimensions, initiatives like ToolLLM and the accompanying ToolBench dataset integrated over 16,000 real-world RESTful APIs. To synthesize reliable training trajectories, ToolLLM deployed a depth-first search-based decision tree (DFSDT) algorithm, enabling the teacher model to rigorously explore multiple reasoning traces, backtrack from execution errors, and establish robust instruction-following data [3]. Similarly, the APIGen framework focuses intensely on verifiable data synthesis. APIGen utilizes hierarchical multi-stage checks—format validation, live execution verification, and semantic consistency checking—to ensure that the synthetic trajectories used to train high-performing models (such as the xLAM family) are logically sound and practically executable [10, 37].

Recent advancements like ToolACE introduce evolutionary diversity into SFT data synthesis. By utilizing a speciation-adaptation-evolution process, ToolACE dynamically generates complex tools across disparate domains, subsequently employing self-guided dialogue generation and dual-layer verification to produce highly complex, multi-agent training corpora [13]. Supervised fine-tuning successfully embeds the "how" of tool execution directly into the neural architecture, allowing highly efficient, smaller-parameter models to consistently outperform much larger generalized models in specific API invocation benchmarks.

Reward-Driven Tool Policy Learning

While SFT excels at instilling the strict syntax of function calling, it fundamentally relies on behavioral cloning. SFT forces the model to mimic static expert trajectories without necessarily imparting the underlying strategic decision-making logic—such as evaluating *when* to invoke a tool versus relying on parametric knowledge, or how to dynamically recover from severe out-of-distribution environmental feedback [11, 25, 33]. Consequently, the state-of-the-art is currently shifting toward Reward-Driven Tool Policy Learning, which models tool use explicitly as a sequential decision-making process optimized via Reinforcement Learning (RL) and Direct

Preference Optimization (DPO).
Traditional RL approaches, such as Proximal Policy Optimization (PPO), encounter severe theoretical hurdles in tool learning. The action space (generating structured JSON or executable code) is highly constrained; random policy exploration rarely yields syntactically valid function calls, resulting in extreme reward sparsity [11, 34]. Furthermore, researchers have identified a critical failure mode termed "structural collapse" during multi-turn agentic RL. In this phenomenon, policy optimization disproportionately amplifies special control tokens, causing the generation to rapidly degenerate into malformed sequences that break the tool-invocation structure entirely, even as the model retains its underlying task competence [33].
To stabilize policy optimization, innovative frameworks introduce specialized reward architectures. The EGPO (Entropy-enhanced Group Relative Policy Optimization) framework, utilized in models like FunRL, integrates the entropy of the model's chain-of-thought reasoning directly into the policy gradient computation. By providing an entropy bonus constrained by a strict clipping mechanism, EGPO encourages the exploration of diverse reasoning strategies without destabilizing the optimization direction, supplemented by rigid binary rewards for syntactic and semantic perfection [11]. Alternatively, frameworks like AutoTraj utilize a sophisticated two-stage recovery process. Rather than discarding failed trajectories, AutoTraj repairs low-quality executions via an LLM-as-Repairer to construct dense preference pairs. It then trains a trajectory-level reward model to provide granular, step-by-step supervision, effectively mitigating the reward sparsity inherent in outcome-only evaluations [34]. Furthermore, the application of information-theoretic metrics, such as Bits-over-Random (BoR), has proven highly effective as an RL reward signal for optimizing retrieval depth, naturally penalizing the inclusion of excessive tools without requiring heavily engineered depth penalties [42].
Simultaneously, DPO has emerged as a highly stable, RL-free alternative for aligning complex tool-use behaviors. By reformulating the reward maximization problem as a direct classification loss over preference pairs, DPO eliminates the architectural overhead of separate reward models and complex hyperparameter tuning [17]. In the context of function calling, frameworks like DiaTool-DPO construct extensive preference datasets by contrasting successful multi-turn tool trajectories against mismatched dialogue flows. This enables the model to natively learn nuanced conversation control—such as accurately judging when to reject a tool call, ask clarifying follow-up questions for missing parameters, or confidently execute the API call—substantially elevating the quality of dynamic user interactions [36, 44].

Autonomous Tool Creation and Procedural Memory

Pushing the theoretical boundaries of artificial agency, recent literature explores enabling LLMs not merely to utilize existing tools, but to autonomously synthesize, test, and deploy novel computational tools. This paradigm effectively shifts the model from a tool-user to a tool-maker [9].
The LATM (Large Language Models As Tool Makers) framework introduces an elegant division of labor. A highly capable, resource-intensive "tool maker" model writes and debugs reusable Python functions specifically tailored to solve novel tasks identified in user prompts. These functions are rigorously verified through automated unit testing and stored in a functional cache. Subsequently, a smaller, highly efficient "tool user" model invokes these newly forged tools to

resolve similar incoming requests [9]. Frameworks like ATLASS build upon this by enabling the agent to autonomously fetch external API documentation and configure virtual environments to dynamically generate tools capable of complex web interactions. By transforming ephemeral problem-solving routines into durable, executable algorithmic artifacts, these frameworks dramatically expand the functional scope of LLMs beyond predefined API ecosystems, establishing a rudimentary but highly effective form of machine-generated procedural memory [23].

Evaluation Benchmarks and Metrics

The rapid proliferation of diverse tool-learning methodologies has necessitated the development of increasingly rigorous, standardized evaluation frameworks. The evaluation landscape has evolved markedly from simple, static format-matching checks to complex, multi-turn, multi-agent simulated environments [16, 23].

API and Function Calling Benchmarks

Foundational benchmarks focus heavily on the syntactic accuracy and zero-shot generalization of discrete function calls. The Berkeley Function-Calling Leaderboard (BFCL) serves as a primary industry standard, evaluating models across thousands of complex test cases spanning Java, Python, and JavaScript APIs. BFCL meticulously segments evaluation into discrete categories: single-turn executions, multi-turn state maintenance, and critical hallucination detection (specifically assessing whether a model inappropriately attempts to invoke a tool when presented with an irrelevant user query) [19].

Similarly, the ToolBench suite evaluates models on their ability to navigate over 16,000 real-world RESTful APIs, explicitly testing depth of reasoning, parallel function invocation, and the model's ability to perform intra-category tool disambiguation [3].

Agentic and Multi-Turn Frameworks

Recognizing that real-world tool use is rarely a solitary, single-step operation, newer benchmarks prioritize simulating dynamic, persistent human-agent interactions. The tau-bench (Tool-Agent-User Benchmark) measures an agent's capability to interact collaboratively with simulated human users and programmatic databases under strict, domain-specific policies (e.g., airline ticketing, retail support) [14]. Unlike static datasets, tau-bench is fundamentally structured as a partially observable Markov decision process. It utilizes a novel "pass^k" metric to rigorously evaluate whether the agent can reliably reach the correct terminal database state across k independent, stochastically varied interaction paths [14].

ACEBench further expands these evaluation dimensions by categorically segmenting testing data into Normal scenarios, Special scenarios (containing highly ambiguous, incomplete, or deliberately erroneous user instructions), and Agent scenarios requiring autonomous, long-horizon multi-agent dialogue [15]. By explicitly assessing how agents recover from erroneous parameters or out-of-scope requests, ACEBench provides a highly granular examination of systemic robustness.

Benchmark	Core Focus	Key Features & Metrics	Interaction Paradigm
BFCL [19]	Function call syntax and execution.	AST parsing, Executable verification, Hallucination detection.	Single-turn & Multi-turn.
ToolBench [3]	Massive API generalization.	16,000+ APIs, DFSDT reasoning evaluation.	Complex Single-turn.
tau-bench [14]	Real-world policy compliance.	Simulates human users, tests database state mutations, uses pass^k.	Dynamic Multi-turn.
ACEBench [15]	Robustness to imperfections.	Normal, Special (erroneous params), and Agent evaluation splits.	Multi-turn & Multi-agent.

Emergent Evaluation Metrics

Evaluation metrics themselves have undergone a paradigm shift, moving from basic textual similarity (e.g., ROUGE, BLEU) to strict functional execution. *Abstract Syntax Tree (AST) Evaluation* algorithmically verifies that the generated tool call strictly matches the predefined schema syntactically, ensuring correct typing and parameter presence without requiring live execution. Conversely, *Executable Evaluation* runs the generated payload against live or deeply mocked endpoints to verify operational correctness, providing a definitive assessment of real-world viability [10, 19].

Challenges and Prospects

While tool-augmented LLMs demonstrate immense potential across diverse verticals, their deep integration with external computational environments introduces profound new vectors of systemic risk and architectural complexity.

Security and Jailbreaking Vulnerabilities

Traditional safety alignment research has overwhelmingly focused on securing the

prompt-to-text generation bottleneck, ensuring models refuse to output toxic or harmful language. However, function calling introduces a structurally distinct execution vulnerability, inadvertently allowing malicious actors to bypass established safety filters entirely.

The *JailbreakFunction* attack paradigm highlights severe alignment discrepancies between standard chat mode and function-calling mode. Researchers have demonstrated that by coercing an LLM to fulfill a malicious request (e.g., writing a sophisticated phishing email or synthesizing malware) by embedding the payload within the string arguments of a seemingly benign function call, attackers can achieve jailbreak success rates exceeding 90% against state-of-the-art commercial models [30, 46]. Because the safety filters are overwhelmingly trained on conversational text, they frequently fail to scrutinize the semantic payload of structured JSON parameters.

Furthermore, autonomous function calling exposes agents to Indirect Prompt Injection (IPI). When an agent utilizes a tool to retrieve untrusted external data—such as scraping a webpage or summarizing an incoming email—adversarial instructions covertly embedded within that external data can hijack the agent's control flow, compelling it to invoke unauthorized data-exfiltration APIs [46]. Multimodal systems confront even stealthier threats; the *Odysseus* attack paradigm employs dual steganography to encode malicious function-calling schemas directly into the imperceptible noise of image inputs. This technique entirely circumvents textual moderation filters, forcing the Multimodal LLM (MLLM) to decode and execute the malicious payload locally [31]. Securing tool-augmented agents demands robust, defense-in-depth measures, including query-tool consistency checkers, cryptographically isolated execution sandboxes, and alignment optimization specifically tuned for structured, agentic outputs [46].

Parameter-Level Tool Unlearning

As the API ecosystem evolves, certain tools inevitably become deprecated, legally restricted, or identified as critical security vulnerabilities. While system developers can superficially remove the tool schema from the model's system prompt, the LLM's parametric memory retains the deep latent knowledge of how to format, reason about, and invoke the deprecated API. This residual memory poses a persistent security and compliance risk, necessitating the development of a novel paradigm: *Tool Unlearning*.

Unlike traditional machine unlearning—which focuses on erasing the memorization of specific training data samples to adhere to privacy regulations (e.g., GDPR)—tool unlearning focuses explicitly on ablating functional, generalized capabilities [12]. Frameworks such as ToolDelete attempt to solve this complex bi-objective optimization problem: maximizing absolute knowledge removal for targeted tools while strictly preserving execution accuracy for retained tools and maintaining general conversational coherence.

Tool unlearning remains exceptionally challenging due to the intricate mechanistic circuitry of LLMs. Advanced metrics like Circuit-guided Unlearning Difficulty (CUD) reveal that the parameters governing function structure are deeply entangled with the model's general reasoning and code-generation capabilities. Easy-to-unlearn samples typically rely on shallow, localized circuits, whereas deeply ingrained tool-use behaviors rely on pervasive, late-stage pathways [45]. Consequently, executing parameter-level unlearning without inducing catastrophic performance degradation across the model remains a highly active and critical area

of investigation [12].

Multimodal Tool Learning

The seamless integration of visual, auditory, and sensory data into tool-learning frameworks represents a critical frontier in artificial general intelligence. MLLMs must not only understand text but must also learn to route complex visual inputs to appropriate analytical tools. However, training these models is severely bottlenecked by the extreme scarcity and prohibitive financial cost of curating human-annotated, multi-step multimodal tool trajectories [32].

Emerging frameworks like SPORT address this data drought by enabling MLLMs to autonomously discover effective tool usage policies through self-exploration. By leveraging step-wise preference optimization, the agent autonomously generates synthetic multimodal tasks, experiments with various visual reasoning tools, and utilizes separate AI-generated feedback to iteratively refine its internal controller policy, entirely eliminating the need for pre-collected human data [32]. Similarly, retrieval architectures are evolving to accommodate multimodal requirements; the HIVE framework utilizes the LLM to explicitly identify logical and visual gaps in initially retrieved text documents, synthesizing highly specific compensatory queries to retrieve supplementary documents that satisfy complex, multi-step visual reasoning tasks [43].

Conclusion

The integration of tool learning and function calling represents a defining watershed in the architectural evolution of Large Language Models. By bridging parametric memory with external computational utilities, LLMs have transitioned from passive repositories of statistical text into active, task-oriented agents capable of interacting with and manipulating the broader digital ecosystem. As demonstrated throughout this comprehensive survey, the field has advanced at a staggering pace across distinct methodological epochs: from the prompt-engineering constraints of Plug-and-Play architectures, through the high-fidelity behavioral cloning of Supervised Tool Learning, and into the current frontier of Reward-Driven Policy Learning via DPO and RL. These advancements are rigorously supported by increasingly sophisticated evaluation frameworks that prioritize dynamic, multi-turn interactions, state-tracking, and systemic robustness over static syntactic verification.

However, the geometric expansion of agentic capabilities inextricably introduces profound new vulnerabilities. The very structural pathways that permit LLMs to seamlessly invoke APIs concurrently serve as vectors for sophisticated jailbreaks, indirect prompt injections, and catastrophic execution loops, necessitating rigorous new paradigms in AI security and precise tool unlearning mechanisms. As the research trajectory accelerates toward reasoning-intensive multimodal integration and autonomous, algorithmic tool creation, resolving the inherent tension between unconstrained agentic exploration and robust, verifiable safety alignment will be paramount. Ultimately, mastering tool learning is not merely a functional enhancement of language models, but an indispensable prerequisite for the realization of versatile, reliable, and secure artificial general intelligence.

References

[1] Qu, C., Dai, S., Wei, X., Cai, H., Wang, S., Yin, D., Xu, J., & Wen, J.-R. (2024). "Tool learning with large language models: a survey." *Frontiers of Computer Science*, 18(6). arXiv:2405.17935.
[2] Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., & Scialom, T. (2023). "Toolformer: Language Models Can Teach Themselves to Use Tools." *Advances in Neural Information Processing Systems*. arXiv:2302.04761.
[3] Qin, Y., Liang, S., Ye, Y., Dang, K., Di, P., Yndurain, Y., ... & Sun, M. (2024). "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs." *International Conference on Learning Representations (ICLR)*. arXiv:2307.16789.
[4] Patil, S. G., Zhang, T., Wang, X., & Gonzalez, J. E. (2023). "Gorilla: Large Language Model Connected with Massive APIs." *Advances in Neural Information Processing Systems*. arXiv:2305.15334.
[5] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." *International Conference on Learning Representations (ICLR)*. arXiv:2210.03629.
[6] Shen, Y., Song, K., Tan, X., Li, D., Lu, W., & Zhuang, Y. (2023). "HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face." *Advances in Neural Information Processing Systems*. arXiv:2303.17580.
[7] Lu, P., Peng, B., Cheng, H., Galley, M., Chang, K.-W., Wu, Y. N., Zhu, S.-C., & Gao, J. (2023). "Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models." *Advances in Neural Information Processing Systems*. arXiv:2304.09842.
[8] Hao, S., Liu, T., Wang, Z., & Hu, Z. (2023). "ToolkenGPT: Augmenting Frozen Language Models with Massive Tools via Tool Embeddings." *Advances in Neural Information Processing Systems*. arXiv:2305.11554.
[9] Cai, T., Wang, X., Ma, T., Chen, X., & Zhou, D. (2023). "Large Language Models as Tool Makers." *International Conference on Learning Representations*. arXiv:2305.17126.
[10] Jiang, Z., Lan, T., Zhu, M., Liu, Z., Hoang, T., Kokane, S., Yao, W., & Tan, J. (2024). "APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets." *Advances in Neural Information Processing Systems*. arXiv:2406.18518.
[11] Hao, B., Wang, M., Xu, Z., Chen, Y., Peng, C., Gu, J., & Zhuang, C. (2025). "Exploring Superior Function Calls via Reinforcement Learning." arXiv:2508.05118.
[12] Cheng, J., & Amiri, H. (2025). "Tool Unlearning for Tool-Augmented LLMs." *International Conference on Machine Learning (ICML)*. arXiv:2502.01083.
[13] Liu, Z., Zhu, M., Lan, T., Hoang, T., Zhang, J., Yao, W., ... & Feng, Y. (2024). "ToolACE: Winning the Function Calling Competition." arXiv:2409.00920.
[14] Yao, S., Chen, Y., Narasimhan, K., & Cao, Y. (2024). "tau-bench: A Benchmark for Tool-Agent-User Interaction." arXiv:2406.12045.
[15] Li, Z., Wang, S., Wang, Y., ... & Chen, X. (2025). "ACEBench: A Comprehensive Benchmark for Assessing Tool Usage." arXiv:2501.12851.
[16] Zheng, Y., Wang, Z., ... & Liu, H. (2026). "Agentic Tool Use in Large Language Models." arXiv:2604.00835.
[17] Rafailov, R., Sharma, A., Mitchell, E., Ermon, C., Manning, C. D., & Finn, C. (2023). "Direct Preference Optimization: Your Language Model is Secretly a Reward Model." *Advances in Neural Information Processing Systems*. arXiv:2305.18290.

[18] Kachuee, M., Ahuja, S., Kumar, V., Xu, P., & Liu, X. (2024). "Improving Tool Retrieval by Leveraging Large Language Models for Query Generation." arXiv:2412.03573.
[19] Yan, T., Patil, S. G., ... & Gonzalez, J. E. (2024). "Berkeley Function-Calling Leaderboard (BFCL)." *UC Berkeley Sky Computing Lab*.
[20] Wang, Y., ... & Chen, Z. (2024). "ToolGen: Unified Tool Retrieval and Calling via Generation." *International Conference on Learning Representations (ICLR)*. arXiv:2410.03439.
[21] Qu, C., ... & Wen, J.-R. (2024). "DRAFT: Iterative Document Refinement for Tool Learning." arXiv:2410.08197.
[22] Shi, Z., Gao, S., Yan, L., Feng, Y., Chen, X., Chen, Z., Yin, D., Verberne, S., & Ren, Z. (2024). "AutoTools: Automating the Tool-Use Workflow." *OpenReview*.
[23] Xia, Y., ... & Li, Y. (2025). "From Selection to Generation: A Survey of LLM-based Active Learning." *Association for Computational Linguistics (ACL)*. arXiv:2502.11767.
[24] Dong, X., ... & Chen, Y. (2024). "A3T: Autonomous Annotation of Agent Trajectories." arXiv:2403.14589.
[25] Zhou, Y., ... & Zhao, H. (2026). "ToolMaster: Trial-and-Execution Paradigm." arXiv:2601.12762.
[26] Tang, Y., ... & Zhang, Z. (2023). "ToolAlpaca: Generalized Tool Learning." *Advances in Neural Information Processing Systems*.
[27] Xu, Z., & Yan, L. (2026). "GenesisFunc: High-quality Function-Calling Data Generation." arXiv:2605.28835.
[28] Huang, Z., ... & Others. (2025). "SkillRouter: Retrieve-and-Rerank Pipeline." arXiv:2603.22455.
[29] Chen, X., ... & Others. (2025). "Tool-DE: Document Expansion for Tool Retrieval." arXiv:2510.22670.
[30] Wu, Y., ... & Zhang, Y. (2026). "JailbreakFunction: Alignment Discrepancies in Tool Use." *Association for Computational Linguistics (ACL)*.
[31] Zhao, X., ... & Wang, L. (2026). "Odysseus: Dual Steganography for Jailbreaking MLLMs." *Network and Distributed System Security Symposium (NDSS)*.
[32] Zhang, Y., ... & Li, Z. (2025). "SPORT: Step-wise Preference Optimization for Multimodal Tool Usage." arXiv:2504.21561.
[33] Wang, H., ... & Chen, X. (2026). "Structural Collapse in Agentic Reinforcement Learning." arXiv:2606.26027.
[34] Li, J., ... & Others. (2026). "AutoTraj: Repairing and Rewarding Tool-Use Trajectories." arXiv:2601.23032.
[35] Miao, Y., ... & Others. (2025). "RITE: Reinforcement Learning for Interleaved Tool Execution." arXiv:2510.11184.
[36] Qiao, S., ... & Others. (2025). "DiaTool-DPO: Dialogue Tool DPO." arXiv:2504.02882.
[37] Zhang, J., ... & Others. (2025). "APIGen-MT: Multi-Turn Data Generation via Simulated Interplay." *Advances in Neural Information Processing Systems*. arXiv:2505.13940.
[38] Liu, H., ... & Others. (2026). "MM-tau-p2: Multi-Modal Benchmark for Dual-Control." arXiv:2603.09643.
[39] Peng, Z., ... & Others. (2024). "ToolRerank: Adaptive and Hierarchy-Aware Reranking." arXiv:2403.06551.

[40] Sun, X., ... & Others. (2025). "LoopTool: Model-Aware Data Evolution Framework." arXiv:2511.09148.
[41] Xia, Y., ... & Others. (2025). "OR-Toolformer: Operations Research Toolformer." arXiv:2510.01253.
[42] Gao, Y., ... & Others. (2026). "MARVEL: Multimodal Adaptive Reasoning-Intensive Retrieval." arXiv:2604.07079.
[43] Chen, Z., ... & Others. (2026). "HIVE: Compensatory Query Synthesis." arXiv:2604.07220.
[44] Wu, C., ... & Others. (2025). "DPO for small LM function calling." arXiv:2410.18890.
[45] Fan, W., ... & Others. (2026). "Circuit-guided Unlearning Difficulty (CUD)." arXiv:2601.09624.
[46] Zhu, Y., ... & Others. (2026). "Indirect Prompt Injection in LLM Agents." arXiv:2603.1023.
[47] Wang, Z., ... & Others. (2024). "Enhancing Function-Calling Capabilities with Decision Tokens." arXiv:2412.01130.